

Архиватора на основе алгоритма Хаффмана

Ратушный Максим Витальевич

Группа: 25.Б43-мм

Кафедра: системного программирования

Научный руководитель: Литвинов Юрий Викторович

Номер семестра практики: 2

1. Введение

Кодирование Хаффмана — классический алгоритм сжатия данных без потерь, применяемый во многих форматах (JPEG, PNG, DEFLATE и др.). В рамках учебной практики была поставлена задача реализовать архиватор на основе алгоритма кодирования Хаффмана с целью получения навыков работы с битовыми потоками ввода-вывода, сериализацией деревьев и восстановлением данных на стороне декодера. Для реализации взят язык Rust.

2. Постановка задачи

Целью работы является получение навыков работы с битовыми потоками ввода-вывода, сериализацией деревьев и восстановлением данных на стороне декодера. Для этого поставлена задача реализовать архиватор на основе алгоритма кодирования Хаффмана.

Для достижения цели необходимо решить следующие задачи:

1. Спроектировать бинарный формат архива, обеспечивающий упаковку иерархии файлов и каталогов в единый поток.
2. Реализовать алгоритм Хаффмана: построение дерева, таблицу кодовых путей, побитовый буфер и функции сжатия/распаковки.
3. Создать интерфейс командной строки (CLI) для упаковки и извлечения.
4. Выполнить функциональное тестирование.
5. Провести экспериментальное сравнение с распространёнными архиваторами (tar+gzip, zip) по времени выполнения и степени сжатия.

3. Обзор

Современные архиваторы обычно комбинируют словарное сжатие (LZ77/LZ78) со статистическим кодированием (Хаффмана или арифметическим). Это позволяет достигать высоких коэффициентов сжатия, но требует сложной реализации. Формат tar+gzip сначала формирует tar-архив (последовательность файлов с метаданными), а затем сжимает его с помощью gzip, использующего DEFLATE (LZ77 + Хаффман) [1][2]. В учебных целях выбран упрощённый подход: файлы объединяются в линейный поток посредством собственного формата (bundling), после чего весь поток сжимается чистым алгоритмом Хаффмана без предварительного LZ-сжатия. Это даёт возможность сосредоточиться на реализации алгоритма Хаффмана, работе с битовыми буферами и проектировании формата.

Язык Rust взят из-за гарантий безопасности памяти [3], удобной системы сборки, богатой экосистемы тестирования, включающей библиотеку proptest для тестирования на основе свойств, позволяющую автоматически проверять инварианты на большом количестве случайных данных, и потенциала для высокой производительности [4].

4. Описание предлагаемого решения

Архитектура программы разделена на четыре модуля:

- `coding` — реализация кодирования Хаффмана: подсчёт частот байтов, построение дерева, таблица кодовых путей `PathTable`, побитовый буфер `BitBuffer` и функции `compress / decompress`.
- `bundling` — бинарный формат архива: последовательность записей, каждая из которых содержит длину относительного пути (`u16` — шестнадцатитбитное беззнаковое число), сам путь (байтовый вектор), флаг «директория» (`u8` — то есть восьмидбитное беззнаковое число; 0 — файл, 1 — директория), и для файлов — длину данных (`u64` — шестидесятитбитное беззнаковое число) и данные. Конец архива маркируется записью с нулевой длиной пути.
- `lib` — интеграция двух предыдущих модулей: рекурсивный обход файловой системы, построение записей `ArchiveEntry` с вычислением относительных путей, а также функции для упаковки и распаковки.
- `main` — CLI, обрабатывающий ключи `-c` (создать архив) и `-x` (извлечь).

4.1. Алгоритм Хаффмана и побитовый ввод-вывод

Сжатие начинается с подсчёта частот всех 256 возможных байтов во входном потоке. Для полученных частот строится бинарное дерево с помощью очереди с приоритетом. Узлы сравниваются по частоте, а при равенстве — по наличию байта (внутренний узел считается меньше листа) и значению байта, что гарантирует детерминированное построение дерева. Если во входных данных присутствует только один уникальный байт, добавляется искусственный лист с нулевой частотой, чтобы дерево оставалось корректным.

Коды Хаффмана представлены не как целые числа фиксированной разрядности (что ограничило бы длину кода 64 битами), а в виде таблицы путей `PathTable`. Для каждого из 256 символов хранятся массив направлений (0 — левый потомок, 1 — правый) длиной до 255 элементов и длина пути. Таблица путей заполняется обходом дерева в глубину.

`BitBuffer` использует восьмидбитный аккумулятор и счётчик накопленных битов. Метод `write_bit` поочерёдно добавляет биты; при заполнении восьми бит младший байт сбрасывается в выходной вектор, что позволяет обрабатывать коды произвольной длины без переполнения. При декодировании заголовков архива (2048 байт частот и восемь байт исходной длины) используется для восстановления дерева, после чего входные биты последовательно проходят по дереву до достижения листа, где извлекается исходный байт. Процесс продолжается до получения заданного количества байт, после чего останавливается, игнорируя выравнивающие биты.

4.2. Формат файла

Формат архива начинается с последовательности записей, каждая из которых начинается с двухбайтовой длины пути, затем идут байты пути и однобайтовый флаг `is_dir`. Если запись является файлом, далее идут восьмибайтовая длина данных и сами данные. Конец потока обозначается записью с длиной пути, равной нулю. Такой подход позволяет хранить произвольное количество файлов и папок с сохранением относительных путей. Благодаря этому при распаковке полностью восстанавливается исходная структура каталогов.

5. Тестирование и эксперименты

5.1. Функциональное тестирование

Для удостоверения в корректности сжатия и формата написаны тесты, проверяющие граничные случаи; также использована библиотека `proptest` для тестирования на основе свойств. Для тестирования модуля `coding` генерируются сотни случайных последовательностей байт длиной до десяти Кбайт; для каждой проверяется тождество `decompress(compress(data)) == data`. Аналогичные тесты написаны для модуля `bundling`: случайный вектор записей `ArchiveEntry` сериализуется и десериализуется, после чего сравниваются поля всех записей. Оба набора тестов не выявили ошибок.

5.2. Экспериментальное сравнение с другими архиваторами

Для оценки эффективности и скорости работы проведены замеры на синтетическом файле `test.txt` и директории с исходным кодом `gcc-releases-gcc-16.2.0`. Сравнивались три инструмента: разработанный архиватор (`huffman_archiver`), `tar+gzip` (`tar.gz`) и `zip`. Для каждого инструмента выполнено шесть прогонов; первый считался прогревочным и отбрасывался, остальные пять усреднялись. Измерялись время сжатия, время распаковки, размер полученного архива и коэффициент сжатия K , равный отношению размеров архива и исходных данных. Все испытания проводились на одной машине (AMD Ryzen 7, 16 ГБ ОЗУ, Linux 6.19.10-zen1-1-zen). Усреднённые показатели приведены в таблице 1. Результаты прогонов можно найти в репозитории проектах.[5]

Таблица 1: Средние значения и стандартные отклонения с поправкой Бесселя (σ) по пяти прогонам (прогревочный исключён)

Файл	Инструмент	Среднее время сжатия	Среднее время распаковки	Размер архива, байт	K
test.txt — время в миллисекундах					
	huffman_archiver	6.7 ± 1.0	1.9 ± 0.6	20656	0.440
	tar.gz	4.29 ± 0.09	3.62 ± 0.09	500	0.011
	zip	1.78 ± 0.09	1.88 ± 0.07	564	0.012
gcc-releases-gcc-16.2.0 — время в секундах					
	huffman_archiver	64.9 ± 4.9	12.124 ± 0.015	604654708	0.718
	tar.gz	20.971 ± 0.029	3.225 ± 0.006	164299087	0.195
	zip	28.30 ± 0.07	11.042 ± 0.008	253628857	0.301

Анализ результатов

На синтетическом файле с повторяющимися последовательностями все три инструмента значительно уменьшают размер, однако `huffman_archiver` показывает коэффициент 0.44, в то время как `tar.gz` и `zip` достигают 0.01. Предположительно, причина в том, что чистое кодирование Хаффмана не способно компактно представить повторяющиеся цепочки символов. На директории с исходными текстами GCC (`gcc-releases-gcc-16.2.0`) `huffman_archiver` даёт относительно небольшое сжатие (0.72), тогда как `tar.gz` достигает коэффициента 0.20, а `zip` — 0.30. Методы LZ77 (используемые в DEFLATE) отлично сжимают дублирующиеся фрагменты, поэтому их результат на порядок лучше.

По производительности на малом файле `huffman_archiver` оказывается самым медленным при сжатии (5.4–7.7 мс против 4.1–4.4 мс у `tar.gz` и 1.7–1.9 мс у `zip`), но при разжатии он сопоставим с `zip` (1.4–2.5 мс против 1.8–2.0 мс) и заметно быстрее `tar.gz` (3.5–3.7 мс). Однако на большом наборе данных сжатие у `huffman_archiver` занимает 61–70 секунд, тогда как `tar.gz` справляется примерно за 21 секунду, а `zip` — примерно за 28 секунд; разжатие занимает около 12 секунд против 3.2 секунды у `tar.gz` и 11 секунд у `zip`.

Разработанный архиватор демонстрирует корректность восстановления данных.

6. Заключение

В ходе выполнения работы получены следующие результаты.

- Реализован консольный архиватор на языке Rust, объединяющий упаковку файлов и каталогов со сжатием алгоритмом Хаффмана.
- Разработан собственный бинарный формат архива с поддержкой иерархии каталогов.
- Реализованы модули для побитового ввода-вывода, построения дерева Хаффмана и таблицы кодовых путей, гарантирующие корректную работу с кодами длиной до 255 бит.
- С помощью тестов подтверждена корректность сжатия и формата.
- Проведено экспериментальное сравнение с `tar.gz` и `zip`, показавшее как ограничения чистого Хаффмана, так и приемлемую скорость работы на небольших данных.

Список литературы

- [1] gzip algorithm specification [Электронный ресурс] // GNU Savannah. — URL: <https://gitweb.git.savannah.gnu.org/gitweb/?p=gzip.git;a=blob;f=algorithm.doc> (дата обращения: 20.06.2026).
- [2] DEFLATE RFC [Электронный ресурс] // RFC Editor. — URL: <https://www.rfc-editor.org/rfc/rfc1951.html> (дата обращения: 20.06.2026).
- [3] Rust in Android: move fast and fix things [Электронный ресурс] // Google Security Blog. — URL: <https://blog.google/security/rust-in-android-move-fast-fix-things> (дата обращения: 20.06.2026).
- [4] Parallel N-Body Performance Comparison: Julia, Rust, and More [Электронный ресурс] // US NSF Public Access Repository. — URL: <https://par.nsf.gov/biblio/10597555> (дата обращения: 20.06.2026).
- [5] Сырые данные по прогонам [Электронный ресурс] // GitHub. — URL: https://github.com/maxicot/huffman_archiver/blob/main/report/raw_data.pdf (дата обращения: 22.08.2026).